CM #1
Présentation
Rappels

TI301 – L2 2024/2025

Nicolas FLASQUE

Présentation du module

Dans la continuité du module TI202 de P1

Vos objectifs

- **Décrire** le comportement des listes chainées, piles et files;
- **Expliquer** les avantages et les inconvénients de chaque structure de données, en termes de complexité temporelle (temps d'exécution) et spatiale (occupation de la mémoire);
- **Justifier** le choix de la structure la plus adaptée à la résolution d'un problème;
- **Implémenter** en langage C les structures de données présentées en utilisant une approche modulaire : couples de fichiers .c/.h, écriture de fonctions;
- **Utiliser** ces implémentations de structures de données en tant qu'API pour décomposer et résoudre des problèmes complexes;
- **Produire** un code correct, documenté et efficace.

Présentation du module

Volumes horaires

- **CM** : 6x2h **TD** : 5x2h **TP** : 2h+3x3h+1h **PRJ** : 2x3h de suivi + 3h soutenance

Outils utilisés

- Le langage utilisé en TD / TP / Projet est le langage C
- Pour les TP, l'environnement utilisé est ZyBooks
- Pour le projet, l'utilisation de Clion est fortement conseillée
- Communication aux promos (deadlines, infos diverses) : Teams et Moodle

Évaluations

1 Devoir Ecrit : **50 %**	
1 note de TD : **20 %**	2 sessions de CC
1 note de TP : **10 %**	1 session de CC 'TP noté'
1 note de PRJ : **20 %**	Soutenance par un enseignant n'ayant pas suivi le groupe

Usages des IA génératives

- ÉTHIQUE
- RESPONSABLE
- PROFITABLE

CONSEILLÉ

Faire des fiches de cours

Créer des quiz pour s'entraîner

Reformuler les messages d'erreur du compilateur

Commenter du code, améliorer du code (lisibilité), analyser du code existant

Vérifier des algorithmes sur lesquels vous avez un doute

Demander des explications ou éclaircissements sur des énoncés

Générer des données de test pour le projet

Obtenir de l'aide technique sur GIT

Toujours avec des explications détaillées !

DECONSEILLÉ

Obtenir des corrigés bruts

0 apprentissage

Produire du code pour les exercices de TP

Ils ne sont pas notés !

Corriger des erreurs sans explication

0 apprentissage

INTERDIT EN PROJET

Générer du code que vous ne comprenez pas à 100 %

Générer des résultats ex nihilo

Cours 1 - listes chaînées simples – rappels et nouveauté

nouveautés

- ✓ Encapsulation (notion de Programmation Orientée Objet)
- ✓ Algorithmes et collections génériques

Pré requis :

Utilisation des structures : définition et accès aux champs
Utilisation des fonctions : prototype, définition, appel, modularité (fichiers .c et .h)
Maitrise des pointeurs : définition, passage de paramètres, allocation dynamique

Encapsulation

Principe : regrouper les données dans des structures intermédiaires

- ✓ Exemple : Column et CdataFrame dans un ancien projet C

- ✓ Un CDataFrame 'est un' ensemble de Column
  - Écriture des fonctionnalités pour le type Column
  - Réutilisation pour le type CDataFrame

Exemple : réutiliser `afficherUneColonne(...)`

```
AfficherUnCDataFrame(...):
    Pour toutes les colonnes
        AfficherUneColonne(...)
```

Application aux listes

Version 'L1'

Structure `MAILLON`

Structure `LISTE`
une `LISTE` est un `MAILLON *`

Version ’P2'

Structure `t_cell`

Structure `t_list`
une `t_list` stocke un `t_cell *` qui est la tête de liste

Traduction en langage C

Version 'L1'

```c
typedef struct maillon {
    int value;
    struct maillon* next;
} MAILLON;
```

renommage

Version 'P2'

```c
typedef struct s_cell {
    int value;
    struct s_cell* next;
} t_cell;
```

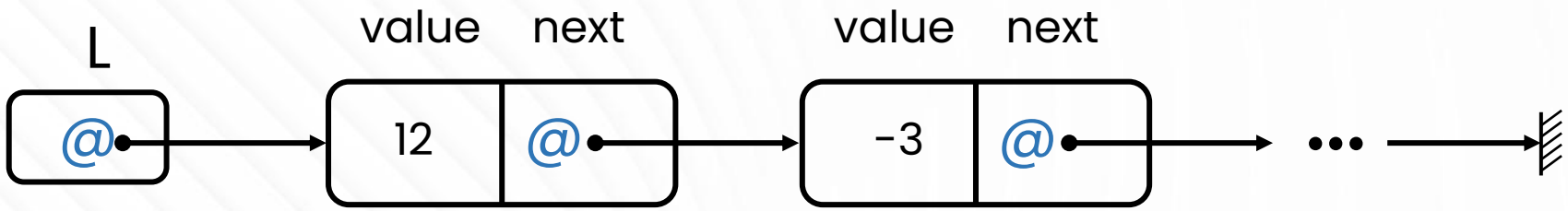
```c
typedef MAILLON* LISTE;
```

nouveau type

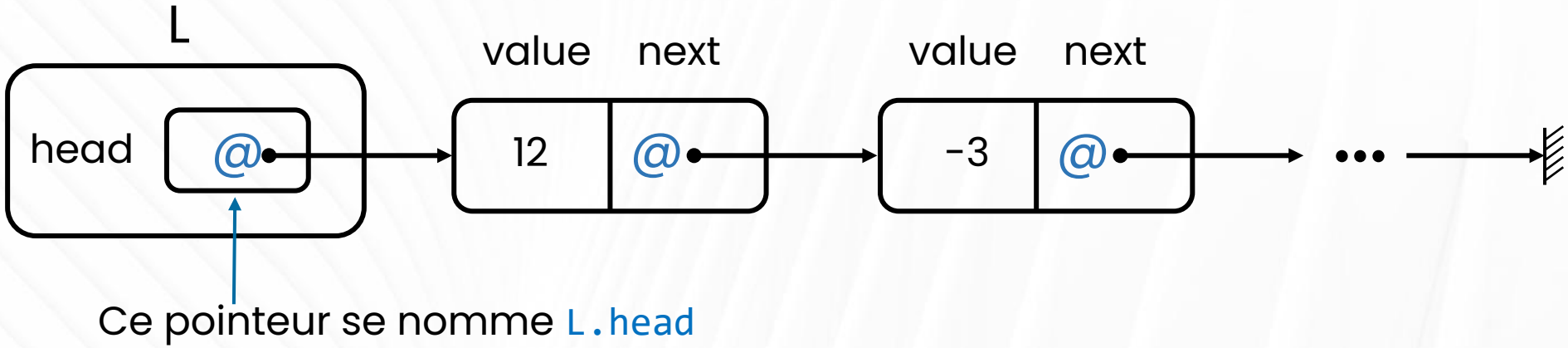
```c
typedef struct s_list {
    t_cell* head;
} t_list;
```

Visualisation d'une telle liste déjà remplie

Version 'L1' `LISTE L;` // et remplissage



Version 'P2' `t_list L;` // et remplissage



A quoi ça sert ?

A priori plus compliqué, mais in fine, plus pratique

La majorité des SDDs s'appuient sur des structures : plusieurs informations simultanées

Ex : gérer un tableau : il faut une taille logique et une taille physique

Ces 3 informations sont systématiquement regroupées => une structure (plus tard, une classe en POO)

Illustration

Un tableau de 'quelque chose' : type générique T

Structure `t_tab`
    valeurs: *tableau de* `T`
    tai_log : *entier*
    tai_phy : *entier*

Les collections

Les Structures de Données sont des collections

Une **collection** regroupe des données (de même type)

Python : listes, tuples, dictionnaires, sets, listes 2D
Langage C : tableaux, listes chaînées
C++ : vector, map

Chaque collection offre des fonctionnalités différentes : implémentation et comportement différents

Le type stocké n'a (presque) pas d'importance

Les algorithmes ne sont pas basés sur ce type

Ex : tri d'un tableau

Même algorithme, pour des `int`, `float`, `char`, `double`, et même `char *`

Le seul besoin : on doit pouvoir **comparer** 2 valeurs de ce type
`<` `>` `=` pour les nombres et les `char`
`strcmp()` pour les chaînes de caractère

Donc on utilise un type quelconque, générique

Classiquement, on le nomme T

Permet d'écrire des algorithmes généraux,

Certains langages le permettent : C++ / Java / C#

Exemple : tri de tableau

```
Algorithme  BubbleSort(Tab : tableau de T, Tai :  entier)

var i, j : entier
Var tmp : T

Pour i de 0 à Tai-2 faire
    Pour j de 0 à Tai-i-2 faire
        Si Tab[j] > Tab[j+1] alors
            tmp ← Tab[j]
            Tab[j] ← Tab[j+1]
            Tab[j-1] ← tmp
        Finsi
    Finpour
Finpour
```

Partie générique en vert,

parties spécifiques en rouge

On peut même calculer sa complexité : ici $O(N^2)$

Application pour les listes chaînées

Définition des structures (algo)

structure t_cell
  value : T
  next : *pointeur sur* t_cell

structure t_list
  head : *pointeur sur* t_cell

Algorithmes génériques pour les listes chaînées

Liste des fonctionnalités

Créer une cellule
Créer une liste (vide)
Ajouter une cellule en tête de liste
Afficher une cellule
Afficher une liste
Rechercher une valeur dans une liste
Indiquer le nombre de cellules dans une liste

Etude des fonctionnalités

Questions essentielles

Périmètre de la fonctionnalité ; **ce qu'elle fait**, ce qu'elle ne fait pas

Ex : 'ajouter une cellule ..' ne fait aucun affichage

Les informations dont elle a besoin : ses **paramètres**

Les informations qu'elle fournit : le **type de retour**

Son **nom** : le plus explicite possible

'Modifier' une structure de données

Questions essentielles

La structure de données est-elle modifiée ?

Soyons précis !

Logique vs technique : exemple du tableau

Soit un tableau Tab stockant des caractères...

Point de vue **logique** : Tab représente l'ensemble des valeurs

Point de vue **machine** : Tab est une variable, donc une valeur unique

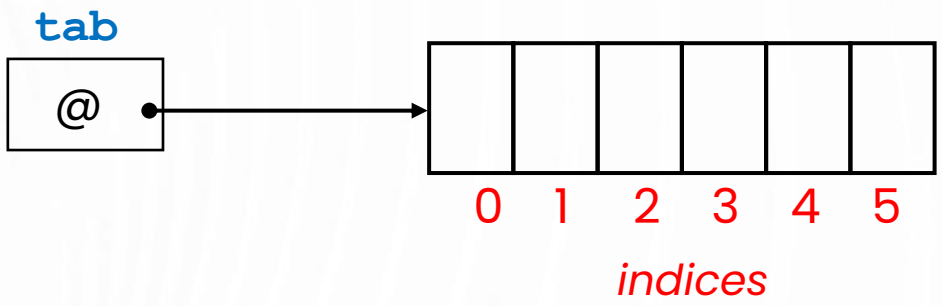
En C : variable tableau = adresse du premier élément stocké

Modifier un tableau = modifier cette adresse

Illustration

Représentation des variables

int tab[6];



Modifier une des valeurs stockées ne modifie pas l'adresse stockée dans la variable **`tab`**

Logique vs technique : exemple de la liste

Soit une liste L stockant des caractères

Point du vue **logique** : L représente l'ensemble des valeurs

Point de vue **machine** : L est une variable, donc une valeur unique

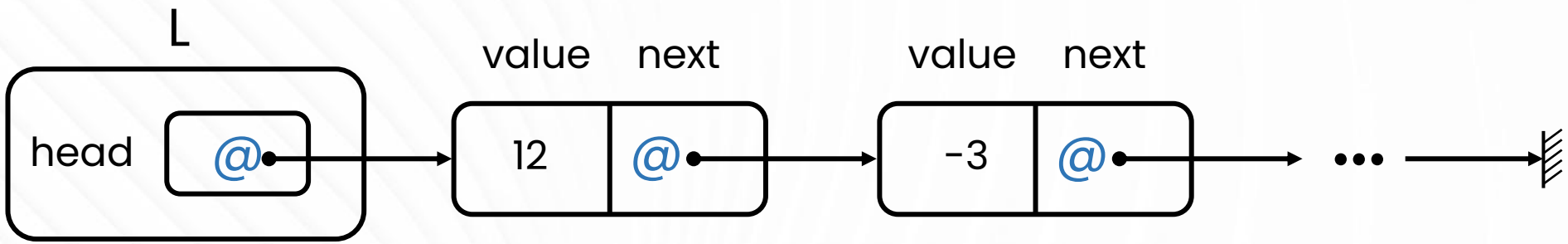
Variable liste = structure stockant l'adresse de la première cellule

Modifier une liste= modifier cette adresse

```
typedef struct s_list {
    t_cell* head;
} t_list;
```

Illustration

Représentation des variables



Modifier **`L`** : modifier **`L.head`** = l'adresse de la première cellule

Par exemple, ajouter une cellule en fin de liste ne modifie pas **`L.head`**

fonctionnalité	nom	paramètre(s)	type de retour
Créer une cellule	`createCell`	T	**Pointeur sur** cellule
Créer une liste vide	`createList`	aucun	liste
Ajouter une cellule en tête de liste	`addCell`	**Pointeur sur** liste, T	aucun
Afficher une cellule	`displayCell`	cellule	aucun
Afficher une liste	`displayList`	liste	aucun
Rechercher une valeur dans une liste	`searchList`	Liste, T	Vrai / Faux
Indiquer le nombre de cellules dans une liste	`countItems`	Liste	entier

Plus en détail

fonctionnalité	nom	paramètre(s)	type de retour
Ajouter une cellule en tête de liste	`addCell`	**Pointeur sur** liste, T	aucun

Pourquoi un pointeur ?

Ajout en tête de liste = la première cellule est une nouvelle cellule

Donc : l'adresse de la première cellule change

Donc **`L.head`** est modifiée

Donc **`L`** est modifié : on doit utiliser **un pointeur**

Règle de modification d'une liste

Pour une liste simplement chaînée

la liste est modifiée ⟺ l'adresse de la première cellule est modifiée

Code C associé

Création d'une cellule

fonctionnalité	nom	paramètre(s)	type de retour
Créer une cellule	`createCell`	T	**Pointeur sur** cellule

```
t_cell *createCell(int val)
{
    t_cell *nouv;
    nouv = (t_cell *)malloc(sizeof(t_cell));
    nouv->value = val;
    nouv->next = NULL;
    return nouv;
}
```

On a choisi le type `int` pour cette fonction, mais on pourrait en choisir un autre

Illustration avec C tutor : **ça se passe ici**

Code C associé

Ajout en tête de liste

Ajouter une cellule en tête de liste	`addCell`	**Pointeur sur** liste, T	aucun

```
void  addCell(t_list *ptr_list, int val)
{
    t_cell *nouv = createCell(val);
    nouv->next = ptr_list->head;
    ptr_list->head= nouv;
    return;
}
```

On a choisi le type `int` pour cette fonction, mais on pourrait en choisir un autre

Code C associé

Créer une liste vide

fonctionnalité	nom	paramètre(s)	type de retour
Créer une liste vide	`createList`	aucun	liste

```
t_list createEmptyList(void)
{
    t_list nouvliste;
    nouvliste.head = NULL;
    return nouvliste;
}
```

Illustration avec C tutor : **ça se passe ici**

Algorithmes avec parcours

Certaines fonctionnalités nécessitent de parcourir la liste

Rappel :

Parcourir un tableau = accéder aux éléments directement via leur 'position' = un indice
Accès **direct** ou **aléatoire**: on peut accéder à l'élément n° 53 directement par exemple

Accéder à un élément quelconque : **Complexité** $O(1)$

Parcourir une liste = partir du premier élément, qui donne accès au deuxième, …
Accès **séquentiel** : Accéder à un élément quelconque : **Complexité** $O(N)$

Exemple : afficher une liste

= afficher tous les éléments d'une liste

Hypothèses : on ne connaît que l'adresse du premier élément

On ne connaît pas le nombre d'éléments stockés

-> donc : boucle **tant que**

```
Tant qu'il y a un élément à afficher
    Afficher l'élément
    Passer à la position suivante
```

Code C associé

Afficher une liste

Afficher une liste	**displayList**	liste	aucun

```
void  displayList(t_list l)
{
    t_cell* curr;

    curr = l.head;

    while (curr != NULL)        // on pointe bien un élément
    {
        printf(format, curr->value); // format : "%d" pour les entiers p.ex
        curr = curr->next ;        // on passe au suivant
    }
}
```

À propos du format

L'algorithme est '**générique**' car il ne dépend pas du type stocké dans les cellules de la liste

En langage C, on doit s'adapter à ce type pour l'instruction

```
printf(format, curr->value);
```

Rappel des formats :

Type stocké dans les cellules	Format à utiliser
`int`	`"%d"`
`char`	`"%c"`
`float`	`"%f"`
`char *`	`"%s"` ou utiliser la fonction `puts()`

Algorithmes associés

Rechercher une valeur dans une liste

Rechercher une valeur dans une liste	`searchList`	Liste, T	Vrai / Faux

Rappel du principe de recherche d'une valeur dans une collection

Au départ : valeur non trouvée (la recherche n'a pas commencé)

Tant qu'on n'a pas trouvé **et** qu'il reste des valeurs dans la collection
    Comparer la valeur cherchée et la valeur dans la collection
    Si elles sont égales
        on a trouvé (et on peut arrêter la recherche)
    Sinon
        Passer à la valeur suivante

Pour la recherche de valeur

fonction vue en TD

Programme vu en TP

De l'utilité des fonctions

Pourquoi créer et utiliser une fonction d'affichage de liste ?

Raison numéro 1

Pour isoler une partie de code réutilisable
Avantages : modularité, flexibilité, aide à la mise au point
Création d'une boîte à outils

Raison numéro 2

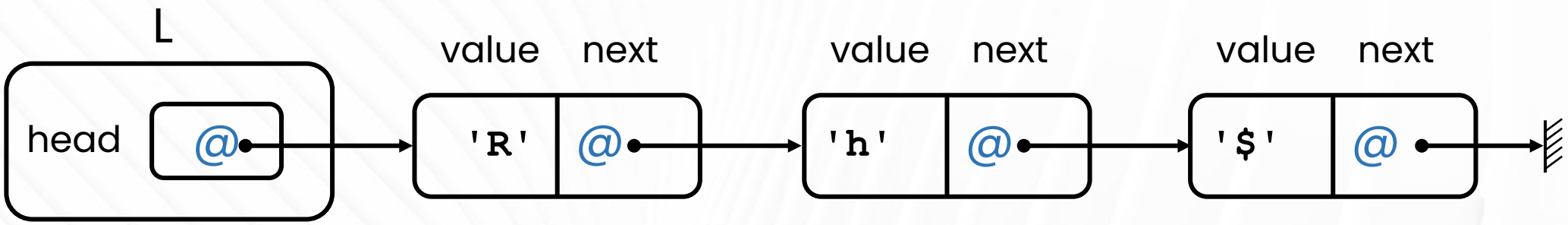
Pour ne pas perdre la tête ☺

Illustration : un programme C qui affiche une liste de manière naïve

Pas de fonction, tout dans le programme principal

On suppose que la liste est initialisée et stocke des caractères

Voici son état :



Que fait la partie de code suivante ?

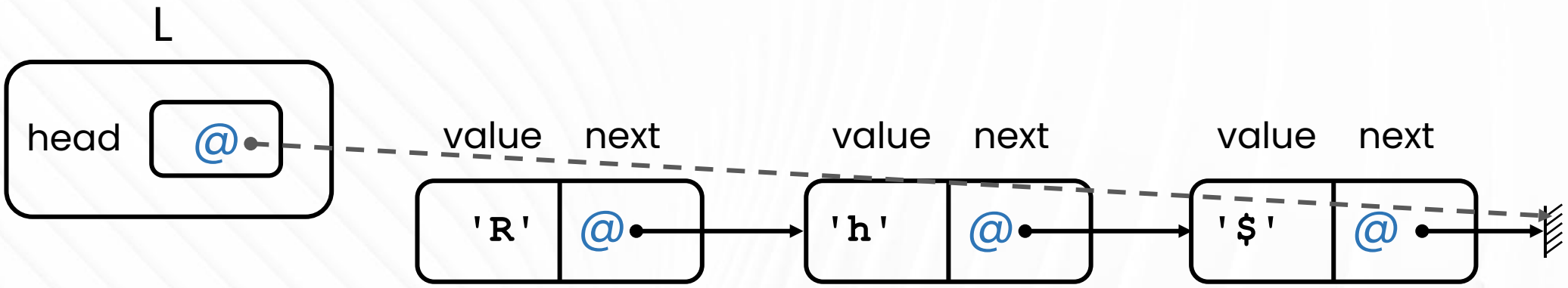
```
int main()
{
    t_list L;
    // initialisation de la liste L puis affichage

    while (L.head != NULL)
    {
        printf("%c", L.head->value);
        L.head = L.head->next;
    }

    return 0;
}
```

On modifie **`L.head`** !

Illustration : Etat final de la liste

On sort de la boucle lorsque L.head vaut NULL

Voici son état :



Le premier élément n'est plus accessible...donc les suivants non plus !

En quoi une fonction résout ce problème ?

Si on donne la liste en paramètre : on obtient, dans la fonction, une copie de l'original, qui ne sera donc pas modifiée !

Illustration avec C Tutor

listes et récursivité

La récursivité s'utilise de manière 'simple' avec les listes

Car : une liste contient un pointeur vers une cellule (son champ **`head`**)

Chaque cellule contient un pointeur vers une cellule

On peut dire que chaque cellule stocke une tête de liste

"Un algorithme pour une liste peut s'appliquer au champ **`'next'`** d'une cellule"

Comment écrire une fonction récursive pour les listes avec le type `t_list` ?

Attention aux types utilisés

Une liste est de type **`t_list`**

Un pointeur vers une cellule est de type **`t_cell *`**

Ces deux types sont distincts.

1) Rédiger la fonction récursive pour le type **`t_cell *`**
2) Rédiger une fonction qui appelle la précédente pour la liste, de type **`t_list`**

Illustration : affichage récursif

On veut réaliser un affichage de liste en utilisant la récursivité

1) On écrit la fonction pour le type **`t_cell *`**

```
void afficheCellRec(t_cell *ptr_cell)
{
        if (ptr_cell != NULL)
        {
                printf(format,ptr_cell->value);
                afficheCellRec(ptr_cell->next); // appel récursif
        }
}
```

C'est bien le type attendu

Illustration : affichage récursif(2)

On veut réaliser un affichage de liste en utilisant la récursivité

1) On écrit la fonction pour le type **`t_list`**

```
void afficheListRec(t_list list)
{
        afficheCellRec(list.head); // démarrage à la tête de liste
}
```

Libérer / détruire une liste

Les éléments sont créés par allocation dynamique

On doit donc libérer l'espace mémoire associé une fois la liste devenue inutile

On procède récursivement

1) Écrire une fonction de libération pour le type **`t_cell *`**

1) Ecrire une fonction qui appelle la première pour la tête de la liste **`t_list`**

Libérer / Détruire une liste (2)

Attention à l'ordre des instructions pour la récursivité

Si on détruit (par `free()`) une cellule pointée, on n'aura plus accès à sa suivante....

Il faut donc accéder à la suivante, **puis** libérer la cellule :

Les libérations se feront en ordre inverse des appels (de la dernière à la première cellule)

fonction vue en TD, programme vu en TP